Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	1-3 Semester (2025-26)
Faculty Name	Mohammed Razia Alangir Banu
Student Name	D.Vignesh Kumar
Roll Number	2520090071
Branch	CSIT
Course Outcome	6

Case Study Topic: Smart Messaging App – Efficient "Compress Chat Messages & Auto-Correct Typos" using Huffman Coding Tree (Non-Linear DS, Greedy) and Edit-Distance Dynamic Programming

Word Done Summary:

In this case study, common practical applications of Non-Linear Data Structures, Greedy Algorithms, and Dynamic Programming were designed, developed, and evaluated using a smart messaging application that compresses chat messages and auto-corrects typed words. Message compression was implemented using Huffman Coding: a min-heap (priority queue) repeatedly extracts the two least-frequent symbols and combines them into a new internal node, building a Binary Tree — a Non-Linear Data Structure — from the bottom up. This is a Greedy Algorithm because at every step it makes the locally optimal choice of merging the two lowest-frequency nodes, which is provably optimal for prefix-free encoding. Traversing the completed Huffman Tree from root to each leaf yields variable-length binary codes that compress the original message far below a fixed-length encoding. Auto-correction of typed words was implemented using Dynamic Programming: the Edit Distance (Levenshtein Distance) between a user's typed word and a dictionary word was computed via a 2D DP table, since simply matching characters one at a time does not guarantee the minimum number of insertions, deletions, and substitutions, while DP explores all alignments through overlapping subproblems to guarantee optimality.

Implementation Details:

1. Min-heap (priority queue) construction over symbol frequencies for Huffman Coding.
2. Greedy merging: repeatedly extract the two lowest-frequency nodes and combine them into a new internal node.
3. Huffman Tree (Binary Tree, Non-Linear Data Structure) construction from the merged nodes.
4. Root-to-leaf traversal of the Huffman Tree to generate prefix codes for each symbol.
5. Compression-ratio comparison between Huffman encoding and fixed-length encoding.
6. Edit Distance Dynamic Programming table dp[i][j] construction between the typed word and a dictionary word.

7. Backtracking through the DP table to identify the minimum-cost sequence of edit operations.
8. Complexity analysis: O(n log n) for Huffman Coding, O(m·n) for Edit Distance DP.

Result:

The Huffman Tree was constructed correctly using the greedy min-heap merging strategy, and the generated prefix codes compressed the sample message to 224 bits, well below the 300 bits required by a fixed 3-bit encoding of the same six symbols. The Edit Distance DP table was built correctly for the auto-correct feature, and backtracking through it confirmed the minimum number of operations needed to transform a typed word into the nearest dictionary word.

Output:

```
Huffman Coding (Greedy + Binary Tree):
Symbol Frequencies : a=45 b=13 c=12 d=16 e=9 f=5
Huffman Codes      : a=0 c=100 b=101 f=1100 e=1101 d=111
Total Encoded Bits : 224 bits   (Fixed-Length Encoding: 300 bits)

Edit Distance (DP) — Auto-Correct:
Typed Word : kitten
Suggested  : sitting
Minimum Edit Operations: 3
```

GitHub Link:

https://github.com/vigneshkumar-stack/DSA-Project

Student Signature	**Faculty Signature**
----------	----------